

01 软件工程基础与项目管理复习讲义

目录

1. 软件工程基础与项目管理	2
1.1 软件是什么	2
1.2 软件的典型特征	2
1.3 软件工程的定义	3
1.4 软件危机与 Ariane 5 案例	3
1.5 SWEBOK 知识域	4
1.6 项目与项目管理	4
1.7 项目管理五大过程组	5
1.8 风险管理	5
1.9 软件质量保证 SQA	6
1.10 软件配置管理 SCM	7
1.11 变更控制流程	7
对应考试模板	8
7.12 配置管理题模板	8
快速背诵清单	9
软件工程	9
项目管理	9

范围:01-01-软件工程基础.pdf、01-02-项目管理基础.pdf; 结合 Exam/Review 中软件工程定义、SQA、SCM、变更控制和风险管理内容。结构: 先整理 PPT 主线, 再放对应考试模板和快速背诵清单。

1. 软件工程基础与项目管理

1.1 软件是什么

软件不是单纯的程序。课程中的核心表达是:

软件 = 程序 + 数据 + 文档 + 知识

IEEE 610.12-1990 对软件的定义强调三类内容:

- 程序
- 规程或过程
- 相关文档与数据

答题时要写出“软件不只等于代码”。代码只是可执行部分, 文档、数据、过程和领域知识共同决定软件能否被理解、运行、维护和演化。

1.2 软件的典型特征

常见概括:

- 无形: 软件没有物理形态, 进度和质量不易直接观察。
- 复杂: 状态、分支、交互和依赖数量高。
- 可复制: 复制成本低, 但开发和维护成本高。
- 易修改: 修改入口多, 但影响传播复杂。
- 退化: 软件不会物理磨损, 但会因环境变化、需求变化和补丁叠加而退化。
- 依赖: 依赖硬件、操作系统、网络、数据库、第三方库和组织流程。
- 智力密集: 核心产出来自设计、抽象、分析和协作。

考试答“软件危机”时, 要把这些特征与项目失败联系起来: 规模变大后, 靠个人经验写代码无法稳定控制质量、进度、成本和维护。

1.3 软件工程的定义

IEEE 定义的关键词：

- 系统化
- 规范化
- 可度量
- 开发、运行和维护

可写为：

软件工程是把系统化、规范化、可度量的方法应用于软件的开发、运行和维护，并研究这些方法的学科。

软件工程解决的不是“怎么写出一段代码”，而是“如何在多人协作、长期演化、质量受控的条件下交付软件系统”。

1.4 软件危机与 Ariane 5 案例

软件危机的标志性节点是 1968 年 NATO 软件工程会议。

常见问题：

- 项目延期
- 成本超支
- 软件质量低
- 难以维护
- 文档缺失
- 需求不断变化
- 开发过程不可控

Ariane 5 案例的答题重点：

- 代码本身来自已有系统，不等于在新环境中正确。

- 隐含的数值范围假设没有被充分记录和验证。
- 复用没有配套完成需求、环境、接口和异常条件验证。
- 失败属于需求假设、集成验证、配置和变更管理问题，不只是编码问题。

1.5 SWEBOK 知识域

传统知识域覆盖需求、设计、构造、测试、维护、配置管理、工程管理、过程、模型方法、质量、职业实践、经济学、计算基础、数学基础、工程基础等。

SWEBOK V4 增强关注：

- 软件体系结构
- 软件工程运维
- 软件安全
- AI for SE：用 AI 辅助软件工程
- SE for AI：用软件工程方法建设 AI 系统

答题时可以用一句话收束：

AI 增强了需求整理、设计草图、代码生成和审查效率，但工程纪律仍由需求验证、接口约束、测试、配置管理和人工评审保证。

1.6 项目与项目管理

PMBOK 中项目的核心特征：

- 临时性：有明确开始和结束。
- 独特性：交付独特产品、服务或成果。

软件项目管理要控制三个基本约束：

- 范围
- 时间
- 成本

三者共同影响质量。范围扩大、时间压缩或成本不足，都会传导到质量风险。

1.7 项目管理五大过程组

按课程表达：

1. 项目启动
2. 项目计划
3. 项目执行
4. 跟踪与控制
5. 项目收尾

常见管理活动：

- 项目计划
- 团队管理
- 成本管理
- 软件质量保证
- 度量
- 过程管理
- 进度管理
- 风险管理
- 配置管理

1.8 风险管理

风险管理循环：

1. 风险识别
2. 风险分析
3. 优先级排序
4. 风险应对

5. 风险监控

常见风险类型：

- 技术风险：新框架、新平台、性能瓶颈、兼容性问题。
- 人员风险：关键成员离开、经验不足、沟通失败。
- 需求风险：需求不清、范围膨胀、利益相关者冲突。
- 进度风险：估算失真、依赖延期、任务拆分不合理。
- 外部风险：政策、供应商、硬件、网络、第三方接口变化。

答题模板：

先列风险，再写影响，再写应对措施。例如：第三方支付接口变更属于外部和技术风险，会影响充值功能上线；应对措施是提前建立接口契约、Mock 服务、回归测试和变更通知机制。

1.9 软件质量保证 SQA

SQA 关注“过程和产品是否满足质量要求”。

常见活动：

- 代码审查
- 静态分析
- 测试
- 过程审计
- 质量度量
- 阶段评审

里程碑质量活动：

- 需求阶段：需求评审、需求度量。
- 体系结构阶段：架构评审、集成测试策略。
- 详细设计阶段：设计评审、设计度量、集成测试准备。
- 构造阶段：代码审查、代码度量、单元测试。
- 测试阶段：缺陷度量、覆盖率分析、回归测试。

1.10 软件配置管理 SCM

SCM 的目标：

- 识别配置项
- 控制变更
- 记录和报告配置状态
- 审计配置
- 保证产品与规格一致

核心术语：

- 配置项 CI：纳入配置管理的项目制品，如需求文档、设计文档、源代码、测试脚本、部署脚本、数据库脚本。
- 基线 Baseline：经过正式评审和批准的规格或产品，后续变更必须走正式变更控制。

SCM 常见活动：

1. 配置项识别
2. 版本管理
3. 变更控制
4. 配置审计
5. 状态报告
6. 发布管理

1.11 变更控制流程

标准作答流程：

1. 提交变更请求 CR。
2. 进行影响分析：影响需求、设计、代码、测试、进度、成本和风险。
3. 进行风险评估。

4. CCB 审查并批准或拒绝。
5. 制定回滚计划。
6. 执行变更。
7. 更新基线。
8. 上线验证。
9. 更新状态报告。

Knight Capital 案例的答题重点：

- 部署遗漏一台服务器。
 - 新旧代码路径并存导致异常交易。
 - 这是发布管理、配置管理和上线验证失败。
 - 技术错误背后是过程控制缺失。
-
-

对应考试模板

7.12 配置管理题模板

题目出现上线后版本不一致、漏部署、旧代码被触发：

问题属于配置管理和发布管理失败。

原因：配置项识别不完整，版本状态未核对，发布前未做配置审计和上线验证。

改进：

1. 将源码、脚本、配置、依赖和部署清单纳入配置项。
 2. 建立发布基线。
 3. 变更走 CR、影响分析、CCB 审批、执行、审计、状态报告。
 4. 发布前核对所有服务器版本。
 5. 发布后执行冒烟测试和回滚预案。
-
-

快速背诵清单

软件工程

- 软件 = 程序 + 数据 + 文档 + 知识。
- 软件工程 = 系统化、规范化、可度量的方法。
- 软件危机来自规模、复杂度、变化和协作失控。
- AI 加速工程活动，但不能替代验证、评审和配置管理。

项目管理

- 项目：临时性、独特性。
- 三约束：范围、时间、成本，影响质量。
- 五过程组：启动、计划、执行、跟踪控制、收尾。
- 风险管理：识别、分析、排序、应对、监控。
- SCM：配置项、基线、版本、变更、审计、状态报告、发布。